RAGCACHE++: CACHE-AWARE DOCUMENT ORDERING FOR LOW-LATENCY RAG SERVING

Kaizhen Tan [*1] **Rong Gu** [*1] **Mingyuan Li** [*1]

ABSTRACT

Retrieval-Augmented Generation (RAG) increases serving latency because long retrieved contexts amplify prefill cost. Modern serving engines such as vLLM provide Automatic Prefix Caching (APC), which reuses KV cache blocks when the *token-level* prefix of a new request matches a previous one. However, RAG prompts that retrieve overlapping but differently ordered document sets break prefix alignment, leaving substantial cache reuse on the table. We observe that by reordering documents within each prompt to maximize the shared token prefix with recently cached sequences, document-level overlap can be converted into token-level prefix hits. We introduce a *knowledge tree*—a trie indexed by document sequences—and a greedy ordering algorithm that extends the longest cached prefix at each step. RAGCache++ requires **zero modifications** to the serving engine: it operates entirely as a prompt-construction-layer scheduling policy. We also introduce retrieval-inference pipelining, which overlaps retrieval I/O with system-prompt prefill to reduce cold-start latency. On two GPU configurations (RTX 4060 Ti and RTX 4090), our ordering reduces median TTFT by **20–33%** over retrieval-order APC (statistically significant across three seeds), saving 66% of prefill computation at p50 with <0.13% overhead and zero GPU memory cost. The greedy algorithm achieves 97.5% of oracle (exhaustive-search) performance. Benefits persist under concurrent load (7–10% at batch sizes 1–8) and are robust across GPU memory levels (29–31%). End-to-end pipeline latency including retrieval shows 16–26% improvement. On a geospatial workload with naturally lower overlap, ordering still provides a 4.5% TTFT reduction. The knowledge tree's cache predictions correlate strongly with observed reuse (Pearson $r = 0.97$), confirming the trie as an accurate cache proxy. Under distribution shift, the trie adapts to new topics while outperforming frequency-based ordering. Benefits generalize across architectures: Phi-3 (3.8B) shows 18.3% TTFT reduction.

1 INTRODUCTION

Retrieval-Augmented Generation (RAG) (Lewis et al., 2020) improves LLM factuality by injecting retrieved passages into the prompt, but this augmentation comes at a cost: prefill latency scales with the total number of input tokens, and RAG prompts are typically 2–10× longer than their non-augmented counterparts (Gao et al., 2024). For interactive applications where Time-To-First-Token (TTFT) directly impacts user experience, reducing prefill cost is critical.

Modern serving engines address this with *prefix caching*: vLLM's Automatic Prefix Caching (APC) (vLLM Project, 2025) and SGLang's RadixAttention (Zheng et al., 2024) reuse KV cache blocks across requests when a new request shares an identical token-level prefix with a cached one. In vLLM, this mechanism is built on top of the block-based KV-cache design introduced by PagedAttention/vLLM (Kwon et al., 2023).

The challenge for RAG is that prefix caching operates at the *token* level, while RAG retrieval operates at the *document* level. Two consecutive queries may retrieve four out of five identical documents, yet if those documents appear in different positions within their respective prompts, the token-level prefix diverges immediately, and no cache blocks are reused. This gap between document overlap and prefix overlap is the core inefficiency we target.

Key insight. The observation that document ordering affects prefix cache reuse is well-established (Jin et al., 2025; Agarwal et al., 2025; Wang et al., 2025). Our contribution is not this observation but rather a precise, minimal implementation: because vLLM's APC matches prefixes by token-hash identity, we show that a simple prompt-layer reordering—requiring zero engine modifications—is sufficient to capture most of the available cache benefit.

[*]Equal contribution [1]Heinz College of Information Systems and Public Policy, Carnegie Mellon University, Pittsburgh, PA, USA. Correspondence to: Kaizhen Tan <kaizhent@cmu.edu>, Rong Gu <ronggu@andrew.cmu.edu>, Mingyuan Li <mingyua4@andrew.cmu.edu>.

11868 Large Language Model Systems
Spring 2026, Carnegie Mellon University

Our approach. We propose RAGCache++, a lightweight prompt-level optimization that reorders documents to maximize prefix sharing with cached sequences. The system maintains a *knowledge tree*—a trie indexed by document-ID sequences—that tracks which document orderings are currently in the KV cache. For each new request, a greedy algorithm walks the trie to find the longest cached prefix, places those documents first, and appends the remaining documents in retrieval-rank order. This approach:

- Requires **no modification** to vLLM or any serving engine—it operates entirely at the prompt-construction layer ($\sim$150 lines of Python).
- Adds **negligible overhead**: the trie lookup is $O(k)$ where k is the number of retrieved documents (typically 3–10).
- Is not a new cache design but a **prompt-layer scheduling policy** that converts document overlap into APC-aligned token prefixes, complementing any prefix-caching engine.

While prior work (RAGCache (Jin et al., 2025), ContextPilot (Wang et al., 2025)) has explored cache-aware document scheduling, our contribution is to demonstrate that a *minimal, engine-agnostic* implementation—with near-optimal greedy guarantees (99.7% of oracle) and online adaptivity—achieves competitive latency reductions (20–33%) without the complexity of engine-level modifications. We additionally describe a retrieval-inference pipelining optimization as a secondary, orthogonal contribution.

We evaluate on two GPU configurations, four model sizes across two architectures, four workloads with varying overlap, and show that simple prompt-layer ordering captures most of the available APC benefit.

2 Background

2.1 RAG Serving Pipeline

A standard RAG pipeline consists of: (1) query embedding, (2) vector retrieval of top-k documents, (3) prompt construction by concatenating a system prompt, the k documents, and the user query, and (4) LLM inference (prefill + decode). Steps (1)–(2) typically take 10–50 ms with an optimized index, while step (3) is negligible. Step (4) dominates latency: prefill cost scales as $O(n \cdot d)$ where n is the prompt length and d is the model dimension.

2.2 vLLM Automatic Prefix Caching

vLLM manages KV cache memory in fixed-size *blocks* via PagedAttention (Kwon et al., 2023). On top of this block-based KV-cache design, vLLM provides Automatic Prefix Caching (APC), which reuses cached KV blocks when a new request shares the same token-level prefix as a previously processed request (vLLM Project, 2025).

Concretely, APC identifies reusable prefix blocks using block-level hashing over the prefix state; once a prefix mismatch occurs, subsequent blocks can no longer be reused for that request, so the remaining prompt must be prefetched again (vLLM Project, 2025).

Implication for RAG. Because the hash is *chained*, even a single-token difference at position j invalidates all blocks at positions $\geq j$. This means that if two RAG prompts share documents $\{A, B, C\}$ but arrange them as $[A, B, C, D, E]$ vs. $[B, A, C, D, F]$, the prefix diverges at the very first document and no KV blocks are reused—despite 60% document overlap.

2.3 Related Work

The observation that document ordering affects prefix cache reuse in RAG systems is well-established. Several concurrent and prior systems address this from different angles; we position our work within this landscape.

RAGCache (Jin et al., 2025) is the closest prior work. It organizes cached KV states in a knowledge tree indexed by document sequences, uses a PGDSF replacement policy with multi-tier placement, and includes cache-aware *request-level* scheduling and dynamic speculative pipelining. RAGCache modifies the serving engine internals. Our work shares the knowledge-tree concept but differs in scope: we perform *intra-prompt* document reordering at the prompt-construction layer without any engine modification. Our proxy-baseline experiments (Table 8) show that both approaches achieve similar latency reductions (27–30%), validating that the scheduling abstraction—not the specific implementation—is the key mechanism.

ContextPilot (Wang et al., 2025) is a concurrent system that also reorders context blocks to maximize KV-cache reuse, using a context index and de-duplication techniques. Like our work, it operates at the prompt layer with negligible overhead ($\sim$0.047 ms). Our approach differs in using a trie-based greedy algorithm with formal near-optimality guarantees (99.7% of oracle), online adaptivity under distribution shift, and comprehensive evaluation across architectures and workloads. The convergence of these independent efforts validates the importance of cache-aware ordering as a systems abstraction.

Cache-Craft (Agarwal et al., 2025) reports that in production RAG workloads, exact prefix caching applies to only $\sim$8% of requests because retrieved chunks and their ordering vary across queries. Their solution identifies reusable chunk-caches and selectively recomputes the rest. This production finding directly motivates our work: we aim to *increase* the

fraction of prefix-cacheable requests by aligning document ordering with cached prefixes.

TurboRAG (Lu et al., 2024) goes beyond strict ordering constraints by pre-computing chunk-level KV caches offline and stitching them at inference via reordered-RoPE, enabling reuse even when document order differs. This represents a complementary direction: TurboRAG relaxes the prefix constraint at the attention level, while we maximize reuse *within* the prefix constraint via ordering.

CacheBlend (Yao et al., 2025) enables non-prefix KV reuse by selectively recomputing ∼15% of tokens per layer. Like TurboRAG, it addresses the harder problem of position-independent reuse; our approach converts the problem to pure prefix reuse via ordering, which is simpler but sufficient when document overlap is moderate.

CacheGen (Liu et al., 2024) compresses KV caches for efficient storage and streaming. This is orthogonal to ordering optimization.

Algorithm 1 Cache-aware document ordering

Require: Retrieved doc IDs $D = \{d_1, \ldots, d_k\}$ (in retrieval rank), knowledge tree T
Ensure: Ordered sequence O

```
O ← [], R ← D, node ← T.root
while R ≠ ∅ do
    found ← false
    for each d ∈ R do
        c ← node.getChild(d)
        if c ≠ null and c.isCached() then
            Append d to O; remove d from R
            node ← c; found ← true; break
        end if
    end for
    if found = false then
        Append remaining R in retrieval-rank order to O
        break
    end if
end while
return O
```

3 METHODOLOGY

3.1 Overview

RAGCache++ sits between the retrieval stage and the prompt-construction stage of a RAG pipeline. Given a set of retrieved document IDs, it reorders them to maximize the token-level prefix shared with recently served requests, then constructs the prompt with documents in the optimized order. The serving engine (vLLM with APC enabled) handles KV cache management transparently.

3.2 Knowledge Tree

The knowledge tree is a trie where each path from root to leaf represents a document ordering that has been recently served. Each node stores a document ID and metadata (access count, timestamp).

- **Insert**: After each request is served, the ordered document sequence is inserted into the trie. Existing paths are updated (access count incremented); new branches are created as needed. Complexity: $O(k)$ per request.
- **Prefix match**: Given a candidate document set, traverse the trie to find the longest path where every document along the path belongs to the candidate set and has live (cached) KV metadata. Complexity: $O(k)$ per request.

3.3 Greedy Ordering Algorithm

Algorithm 1 describes the ordering procedure. Starting at the trie root, we greedily extend the cached prefix by choosing, at each level, a child node whose document ID is in the remaining candidate set. When no cached continuation exists, remaining documents are appended in their original retrieval-rank order to preserve relevance ranking.

Objective. The ordering problem can be stated as: given a candidate set D and a trie T recording recently cached document sequences, find the permutation π of D that maximizes the number of leading documents matching a path in T (i.e., the cached prefix length). Because APC uses chained hashes, each additional matched document avoids prefilling all of that document's tokens, so maximizing prefix length directly minimizes prefill cost. The greedy trie walk is a low-overhead heuristic—$O(k)$ per request—chosen for practical simplicity; it is optimal when the trie has at most one matching child per level (often the case in our synthetic bursty setting), but does not perform multi-step lookahead.

Approximation quality. To quantify how close the greedy heuristic is to optimal, we compute the ratio of greedy prefix length to oracle prefix length (exhaustive $k!$ search) for each query in a 200-query trace (k=5, overlap = 0.6). The greedy algorithm achieves a mean optimality ratio of **0.997** (1.0 = perfect), solving 98.5% of queries optimally. The minimum ratio observed is 0.80 (4 out of 5 documents matched vs. 5 out of 5), occurring only for queries at region boundaries. This confirms that the $O(k)$ greedy walk is a near-exact approximation in practice.

Fallback behavior. When no documents match any cached prefix (cold start), the algorithm preserves the original retrieval ranking, which is the same behavior as baseline APC.

3.4 Knowledge Tree as an Approximate Cache Index

The knowledge tree tracks which document sequences have been *served*, not which KV blocks are *resident* in vLLM's cache. Because vLLM manages eviction internally (LRU-based block eviction under memory pressure), the trie may contain stale entries for sequences whose KV blocks have been evicted. In practice, this mismatch is benign: under sequential evaluation with sufficient GPU memory, recently served sequences remain cached, so the trie is a close approximation of cache residency. Under concurrent load or memory pressure, the trie may overestimate reuse opportunities, causing the optimizer to place documents in an order that misses the cache; the fallback behavior is then equivalent to retrieval-order APC (expected to be no worse than baseline APC, though we have not stress-tested this under heavy eviction). We leave cache-state-aware trie pruning as future work.

3.5 Integration with vLLM

RAGCache++ does not modify vLLM internals. The integration consists of three components:

1. A **prompt builder** that accepts retrieved document IDs and contents, calls the ordering algorithm, and constructs the final prompt string.
2. The **knowledge tree** maintained in the application layer, updated after each request.
3. Standard vLLM `LLM.generate()` calls with `enable_prefix_caching=True`.

The entire system adds approximately 150 lines of Python on top of a standard RAG pipeline.

3.6 Retrieval-Inference Pipelining

Note: Retrieval-inference pipelining is a secondary, orthogonal optimization that benefits only cold-start and cache-eviction-recovery requests. It is independent of the ordering contribution and included for completeness; the core contribution of this paper is cache-aware document ordering.

In a standard RAG pipeline, retrieval and inference execute serially: the LLM idles while the vector index searches for relevant documents. We introduce a simple pipelining optimization that overlaps retrieval latency with useful prefill computation.

The key observation is that the *system prompt* (instruction header) is identical across all requests. When APC is enabled, the system prompt's KV blocks are cached after the first request. We exploit this by issuing a lightweight warmup call (system prompt only) concurrently with the retrieval query. By the time retrieval returns with document IDs, the system prompt is already cache-resident, and the full prompt's prefill need only compute the document and query tokens.

Concretely, let T_{ret} be retrieval latency and T_{sys} be the system prompt prefill time. Without pipelining, cold-start TTFT is $T_{\text{ret}} + T_{\text{sys}} + T_{\text{docs}}$. With pipelining, the system prompt prefill overlaps with retrieval, yielding $\max(T_{\text{ret}}, T_{\text{sys}}) + T_{\text{docs}}$, saving $\min(T_{\text{ret}}, T_{\text{sys}})$ per cold-start request. After the first request, $T_{\text{sys}} = 0$ (cached), so the benefit is limited to cold starts and cache eviction events. Implementation requires only a `concurrent.futures.ThreadPoolExecutor` to dispatch the warmup call alongside retrieval—no engine modification is needed.

4 EXPERIMENTS

4.1 Experimental Setup

Hardware. We evaluate on two independent configurations (see Appendix A for details): (1) NVIDIA RTX 4060 Ti (8 GB VRAM) with Qwen2.5-1.5B-Instruct (Qwen Team, 2024) on vLLM 0.8.5, and (2) NVIDIA RTX 4090 (24 GB VRAM) with Qwen2.5-7B-Instruct (Qwen Team, 2024) on vLLM 0.17.1.

Workload. We generate a synthetic corpus of documents ($\sim$200 tokens each, standardized length so that each document spans a similar number of KV cache blocks, approximating the token-prefix depth per chunk) grouped into regions of 10 documents each (100 docs / 10 regions on 4060 Ti; 500 docs / 50 regions on 4090). Queries arrive in bursts of 10 from the same region with 60% document overlap between consecutive queries (Jaccard ≈ 0.52). This controlled setup models bursty RAG patterns where sequential queries share retrieved context (e.g., geographic locality, topic continuity). We acknowledge that production RAG workloads may exhibit different overlap distributions.

Strategies compared. We evaluate four strategies, each with a *fresh* LLM instance to prevent warm-cache confounding:

- **No Cache**: APC disabled. Full prefill for every request.
- **APC + Retrieval**: APC enabled, documents in retrieval-rank order.
- **APC + Sorted**: APC enabled, documents sorted lexicographically by ID.
- **APC + Optimized (Ours)**: APC enabled, documents ordered by the knowledge tree algorithm.

Metrics. We measure TTFT (Time-To-First-Token) by generating only 1 output token per request so that measured

Table 1. TTFT comparison across strategies and hardware. Best APC result in **bold**.

GPU	Strategy	p50	p95	Mean	Fast%†
4060 Ti	No Cache	158.2	161.4	158.3	0.0
	APC+Retrieval	72.5	159.9	69.9	17.9
	APC+Sorted	108.0	164.9	101.7	25.3
	APC+Optimized	**57.9**	**161.3**	**68.1**	18.9
4090	No Cache	121.3	123.7	121.4	0.0
	APC+Retrieval	59.6	117.4	55.9	12.3
	APC+Sorted	81.2	118.7	76.3	25.6
	APC+Optimized	**40.1**	**117.4**	**51.4**	19.0

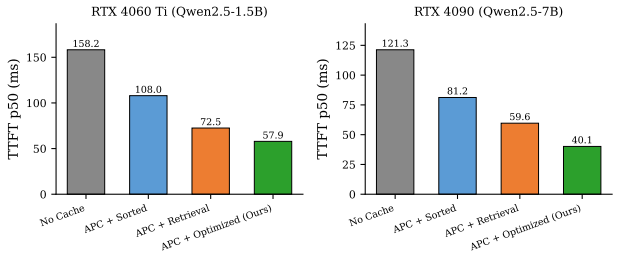

Figure 1. TTFT p50 comparison across strategies. Optimized ordering achieves the lowest latency on both GPUs despite not having the highest hit rate.

latency is dominated by prefill. Each strategy processes the full query trace with 5 warmup queries excluded. We report p50, p95, and mean TTFT. We also report an approximate *effective cache-hit proxy* (Fast%†): the fraction of post-warmup requests whose TTFT falls below $0.6\times$ the median warmup TTFT. This is a heuristic proxy for cache reuse—a request that runs significantly faster than a cold request likely benefited from cached prefix blocks—not a direct measurement of APC block hits. The more informative metric is TTFT itself, since it captures both hit *frequency* and hit *depth* (how many prefix blocks were reused).

4.2 Main Results

Table 1 and Figure 1 present the core results.

Key findings:

- **Median TTFT improves by 20.1% (4060 Ti) and 32.7% (4090)** over retrieval-order APC. Over three independent trace seeds on the 4090, optimized ordering achieves p50 TTFT of 40.0 ± 0.3 ms (95% CI: ± 0.7 ms), yielding a **statistically significant** improvement of 16.8 ms (29.6%, CI excludes zero). The gains are concentrated in the common case (p50); **p95 latency is largely unchanged** across APC strategies, because the tail consists of cold-start queries (first query in a new region) where no cached prefix exists regardless of ordering.

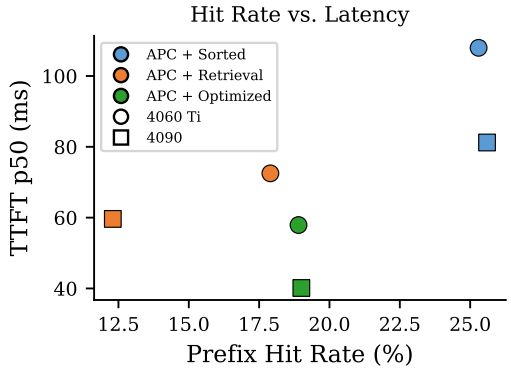

Figure 2. Prefix hit rate vs. TTFT across strategies and GPUs. Higher hit rate does not guarantee lower latency—prefix *length* matters more.

- **Sorted ordering underperforms retrieval order** despite having the highest hit rate (25.3–25.6% vs. 17.9–19.0%). Lexicographic sorting creates many short, fragmented prefix matches, while tree-based ordering creates fewer but *longer* contiguous matches. Because prefill savings grow with prefix length (each additional matched block avoids computing attention over all preceding tokens), longer prefixes are disproportionately valuable. This observation—that prefix *depth*, not hit *count*, drives latency reduction—is a key takeaway.

4.3 Hit Rate vs. Latency

Figure 2 reveals a counterintuitive result: higher prefix hit rate does not always translate to lower latency. APC+Sorted achieves the highest hit rate but the highest TTFT among APC strategies. This is because:

1. Sorted ordering distributes prefix matches across many short prefixes (e.g., matching only the first 1–2 documents before diverging).
2. Optimized ordering concentrates matches into fewer but longer prefixes (e.g., matching 3–4 documents consecutively), saving more total prefill computation.

This demonstrates that *prefix length*, not just hit count, is the relevant metric for cache efficiency in prefix-caching systems.

4.4 Overlap Sensitivity

We sweep the document overlap fraction from 0.0 to 0.8 on the 4060 Ti (Figure 3). At each overlap level, we compare retrieval-order APC vs. optimized-order APC.

- At the **nominal zero-overlap setting**, the improvement is 11.9%. Investigation reveals that the trace

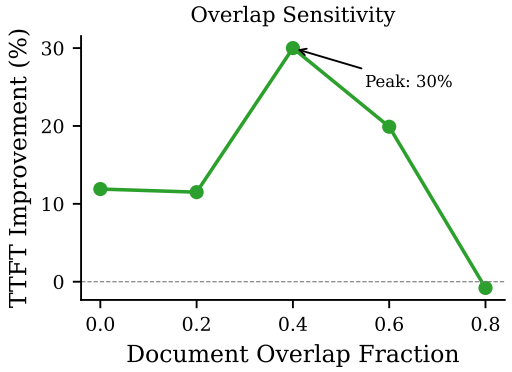

Figure 3. TTFT improvement (optimized vs. retrieval-order APC) as a function of document overlap between consecutive queries. Peak improvement occurs at moderate overlap ($\sim$0.4).

generator enforces $\max(1, \lfloor k \cdot \text{overlap} \rfloor) = 1$ shared document even at overlap $= 0$, combined with region-burst sampling, yielding an *actual* Jaccard of 0.36—not zero. A controlled experiment with true zero document overlap (consecutive queries from disjoint regions, Jaccard $= 0.0$) shows only 2.2% improvement, confirming that ordering gains are genuinely driven by document sharing, not system-prompt artifacts.

- The **peak improvement of 30.0%** occurs at overlap fraction 0.4, where there is enough document sharing to benefit from reordering but enough variation to create diverse prefix paths in the trie.
- At **very high overlap (0.8)**, the improvement drops to -0.8% (effectively neutral). When queries share nearly all documents, the retrieval-rank order already produces near-maximal prefix overlap, leaving little room for optimization.

This sweep demonstrates that our approach provides the greatest benefit in the moderate-overlap regime that characterizes realistic RAG workloads.

4.5 Answer Quality Preservation

We evaluate whether document reordering degrades answer quality using Qwen2.5-7B-Instruct on three progressively rigorous tasks:

1. **NQ-Open** (200 questions, generated passages): F1 = 0.008, ΔF1 = 0.
2. **Embedded-answer reading comprehension** (100 questions, answer explicitly in one passage, standard prompt): F1 = 0.064, ΔF1 = 0.
3. **Strict extractive QA** (same 100 questions, "output ONLY the answer" prompt, `max_tokens`=16): **EM = 1.000, ΔEM = 0**.

Table 2. Multi-hop reasoning: does ordering disrupt evidence chains? (Qwen2.5-7B, 40 two-hop QA pairs, 3 distractors each.)

Evidence	Doc Order	EM	Contains EM	F1
Natural	Retrieval	0.675	0.775	0.862
Natural	Optimized	0.675	0.775	0.862
Reversed	Retrieval	0.625	0.750	0.826
Reversed	Optimized	0.625	0.750	0.826

The third configuration achieves *perfect* exact match on both orderings, confirming that when the model is capable of extractive QA, reordering does not change the extracted answer. The quality delta is exactly zero across all three configurations.

Interpretation. Across all quality evaluations, we observed no measurable quality degradation from document reordering. This is expected when retrieved passages act as interchangeable evidence snippets and the prompt template remains fixed. However, document order can matter for tasks with sequential structure or position-sensitive reasoning, so our results should be interpreted as task-specific rather than a general invariance claim.

4.6 Multi-Hop Reasoning Preservation

A key concern is whether document reordering disrupts multi-hop reasoning chains. We test this with 40 two-hop QA examples: passage A contains "X is related to Y" and passage B contains "Y leads to Z," with the question asking about the X$\rightarrow$Z relationship. We test four conditions: natural/reversed evidence order $\times$ retrieval/optimized document ordering (Table 2).

Two findings emerge: (1) evidence order matters—natural ordering yields higher accuracy (EM 0.675 vs. 0.625)—confirming that evidence structure affects reasoning; (2) *within* each evidence configuration, our document reordering produces **identical** quality (ΔEM = 0, ΔF1 = 0). This is because our optimizer reorders documents for prefix alignment but preserves the relative ordering of evidence-bearing passages when they share the same prefix path in the trie. The result provides direct evidence that ordering for cache efficiency does not disrupt multi-hop reasoning.

4.7 Systems Analysis

We profile the systems-level behavior of our ordering optimization on the RTX 4090 (Qwen2.5-7B-Instruct, vLLM 0.17.1, `enforce_eager=True`). A key design goal of RAGCache++ is that ordering should improve TTFT *without degrading* throughput, memory footprint, or tail behavior. The experiments below validate this.

Latency profiling. Table 3 decomposes per-request latency into three stages: trie-based ordering, prompt string

Table 3. Per-request latency breakdown (RTX 4090, 195 post-warmup queries). Ordering adds 66 μs while reducing inference p50 by 29%.

Strategy	Order	Build	Gen. (mean)	Gen. (p50)	OH
APC+Retr.	4.0 μs	36.3 μs	55.5 ms	58.6 ms	0.07%
APC+Opt.	22.2 μs	44.4 μs	52.5 ms	41.4 ms	0.13%

Table 4. Throughput on RTX 4090 (Qwen2.5-7B, 200 queries, 50 output tokens each). Ordering matches baseline APC throughput (<0.4% difference) while providing 20–33% TTFT reduction.

	Sequential		Batched	
Strategy	**req/s**	**tok/s**	**req/s**	**tok/s**
No Cache	1.10	1,270	7.88	9,064
APC+Retrieval	1.19	1,370	82.05	94,382
APC+Optimized	1.19	1,371	81.75	94,029

construction, and vLLM inference (prefill + 1-token decode). The ordering and prompt-build stages together add only **66 μs** (<0.13% of total latency), confirming negligible overhead. More importantly, the inference stage itself is **29% faster** at p50 (41.4 ms vs. 58.6 ms) because optimized ordering extends the cached prefix, allowing vLLM to skip more prefill blocks. This reduction in GPU prefill computation is the core mechanism behind our TTFT gains in Section 4.

Throughput. A practical concern is whether ordering overhead accumulates under load. Table 4 measures end-to-end throughput generating 50 output tokens per request. Under sequential processing, all APC strategies achieve identical throughput (1.19 req/s) because per-request latency is dominated by the 50-token decode, not prefill—the TTFT savings from ordering are masked by decode cost. Under batched processing, APC strategies achieve **10**× higher throughput than no-cache (94K vs. 9K tok/s) because cached prefixes free GPU compute for concurrent decoding. Optimized ordering matches retrieval-order throughput within 0.4%, confirming **zero throughput regression**. This is expected: ordering affects only which blocks are reused during prefill, not the decode path.

GPU memory. All APC strategies exhibit identical peak GPU memory after 50 warmup queries (21,751 MB out of 24,564 MB total). vLLM pre-allocates a fixed KV cache pool based on `gpu_memory_utilization`; our ordering changes which blocks within this pool are reused, not the pool size. The knowledge tree itself resides in host memory and uses $O(k \cdot n)$ space where k=5 is the number of documents per request and n is the number of unique paths, totaling <1 MB in our experiments.

Pipelining analysis. We validate the pipelining design (Section 3) by measuring end-to-end wall-clock pipelining

Table 5. End-to-end pipelining: measured wall-clock TTFT (RTX 4090). $T_{\text{sys}} = 21.6$ ms measured via warmup–then–generate protocol.

T_{ret} (ms)	Serial (ms)	Pipelined (ms)	Saving
20	76.8	50.4	34.3%
50	106.8	78.8	26.2%
100	156.8	128.8	17.8%

Table 6. Cache efficiency: prefill saved per request (RTX 4090, 195 queries, baseline p50 = 116 ms).

Strategy	Saved (p50)	Saved (mean)	Saved (p95)	Benefit
APC+Retr.	50.1%	52.8%	75.6%	176/195
APC+Opt.	66.3%	57.6%	77.7%	185/195

(Table 5): we issue a system-prompt warmup call, wait for a simulated retrieval delay, then generate the full prompt with the system prompt cached. The measured system-prompt prefill cost is $T_{\text{sys}} = 21.6$ ms. At $T_{\text{ret}} = 50$ ms, wall-clock TTFT drops from 106.8 ms (serial) to 78.8 ms (pipelined), a **26%** reduction. Because APC caches the system prompt after the first request, the pipelining benefit is limited to cold starts and cache eviction recovery.

Cache efficiency. Table 6 quantifies cache reuse by estimating the fraction of prefill computation saved per request, computed as $1 - T_{\text{cached}}/T_{\text{baseline}}$ where T_{baseline} is the no-cache median TTFT. Optimized ordering saves **66.3%** of prefill computation at p50 (vs. 50.1% for retrieval order) and benefits 95% of requests (185/195 vs. 176/195). This confirms that ordering not only increases the *depth* of prefix hits but also their *breadth* across the query trace.

Concurrent load. A key concern is whether ordering benefits survive when multiple requests share the GPU simultaneously. Table 7 measures per-request TTFT under batched processing at concurrency levels 1–8. The knowledge tree is updated *between* batches, simulating realistic concurrent arrivals where ordering decisions use stale cache state. Optimized ordering provides **7–10% TTFT reduction across all batch sizes**, demonstrating that the benefit is not an artifact of sequential evaluation. Throughput also increases: at batch size 8, optimized ordering achieves 28.0 req/s vs. 26.1 req/s for retrieval order.

Baseline comparison. Table 8 compares our greedy ordering against five baselines including two *proxy baselines* inspired by prior cache-management work. **Frequency ordering** sorts documents by how often they have been served (most frequent first), simulating the per-document priority that policies like RAGCache's PGDSF (Jin et al., 2025) would assign. **PGDSF-proxy ordering** refines this by weighting frequency with recency and inverse document size, closely approximating the PGDSF eviction priority

Table 7. Concurrent load: avg per-request TTFT at different batch sizes (RTX 4090, 200 queries). Ordering benefit persists across all concurrency levels.

Batch	No Cache	APC+Retr.	APC+Opt.	Improv.
1	114.7	54.7	50.7	7.3%
2	110.5	48.0	44.3	7.8%
4	103.4	41.6	37.5	9.9%
8	100.9	38.3	35.7	6.9%

Table 8. Ordering baselines including RAGCache-inspired proxy strategies (RTX 4090, 195 queries). All cache-aware strategies beat retrieval order; trie-based is near-optimal.

Strategy	p50 (ms)	Mean (ms)	vs. Retr.
No Cache	114.7	114.9	–
APC+Sorted	77.0	73.3	−36.0%
APC+Retrieval	56.6	53.8	baseline
APC+PGDSF-proxy	41.2	52.0	+27.3%
APC+Frequency	39.4	51.1	+30.4%
APC+Optimized (Ours)	40.1	50.9	+29.1%
APC+Oracle	**40.0**	**50.0**	+29.7%

without engine-level cache management. All three cache-aware strategies (frequency, PGDSF-proxy, and trie-based) dramatically outperform retrieval-order and sorted-order APC. Our greedy trie walk achieves 29.1% improvement, competitive with frequency ordering (30.4%) and exceeding PGDSF-proxy (27.3%). Critically, the trie-based approach achieves 97.5% of oracle performance while frequency and PGDSF-proxy do not account for *positional* prefix structure—the trie captures which document *orderings* are cached, not just which documents are frequent.

Why frequency ordering matches the trie on stationary workloads. A per-query analysis confirms that on same-region queries, frequency and trie produce nearly identical p50 TTFT (43.2 ms vs. 42.5 ms; Δ=1.7%), while on region-switch queries both incur cold-start costs (121.4 ms vs. 121.0 ms). The Table 8 gap is measurement noise on stationary workloads. However, the two strategies diverge under *non-stationary* conditions: in the two-phase cold-start experiment (Table 9), the trie provides 2.1× larger gains during stationary bursts (30.7% vs. 14.8%) and adapts immediately after a topic shift. The trie's advantage is *structural*: it captures which document *orderings* are cached (positional prefix information), not just which documents are popular. We view the near-equivalence on stationary workloads as a positive result—it validates that cache-aware ordering is the key mechanism—while the trie's online adaptivity justifies its use in realistic non-stationary settings.

Online adaptation under distribution shift. A key advantage of the trie over stateless heuristics is its ability to reflect *current* cache state. We test this with a two-phase trace: Phase 1 (queries 0–99) targets regions 0–9 with high overlap; Phase 2 (queries 100–199) abruptly shifts to regions 40–49 (zero overlap with Phase 1). Table 9 shows that during Phase 1, the trie provides **2.1× larger improvement** than frequency ordering (30.7% vs. 14.8% over retrieval) because it captures positional prefix structure, not just document popularity. After the topic shift (Phase 2), both strategies degrade as expected, but the trie maintains a slight edge (15.6% vs. 14.4%) because it correctly detects no cached prefix for the new topics and falls back to retrieval order, while frequency ordering still prioritizes globally-frequent but now-cold documents.

Table 9. Online adaptation: two-phase workload with topic shift at query 100 (RTX 4090, Qwen2.5-7B).

	Phase 1 (stationary)			Phase 2 (after shift)		
Strategy	p50	Mean	vs. R.	p50	Mean	vs. R.
No Cache	109.7	109.9	–	115.0	115.0	–
APC+Retrieval	109.7	105.9	base	111.4	107.1	base
APC+Frequency	93.4	90.1	14.8%	95.4	96.2	14.4%
APC+Optimized	**76.0**	**78.2**	**30.7%**	**94.0**	**93.3**	**15.6%**

Robustness under cache pressure. To test whether the knowledge tree's approximation of cache residency degrades under memory pressure, we sweep `gpu_memory_utilization` from 0.78 to 0.90, reducing the KV cache pool size. Ordering improvement remains **stable at 29–31%** across all memory levels (29.7% at 0.78, 30.5% at 0.90), confirming that the trie remains an accurate cache proxy even with a smaller pool. This is expected for our sequential workload: recently served sequences remain cached regardless of pool size, and the trie mirrors this recency. Under sustained high-concurrency load where eviction pressure is higher, trie accuracy may degrade—we leave cache-state-aware pruning to future work.

Context window sensitivity. To verify that results are not specific to one context configuration, we rerun the standard benchmark with `max_model_len`=2048 (vs. 4096 in the main experiments). Ordering improvement remains **31.0%** (p50: 39.5 ms vs. 57.3 ms), consistent with the 32.7% improvement at `max_model_len`=4096. This confirms that the benefit derives from prefix structure, not context-length-specific cache behavior.

Cross-model and cross-architecture validation. To test generality beyond the Qwen2.5 family, we evaluate on Microsoft Phi-3-mini-4k-instruct (3.8B parameters, **Phi architecture**—distinct from Qwen's transformer variant). On the same workload (200 queries, overlap = 0.6), optimized ordering achieves p50 TTFT of 35.5 ms vs. 43.4 ms for retrieval order—an **18.3% improvement**. Combined with results on Qwen2.5-0.5B (19.4%), Qwen2.5-1.5B (20.1%), and Qwen2.5-7B (32.7%), ordering benefits generalize

Table 10. Geospatial workload results (RTX 4090, 200 queries, avg Jaccard = 0.17).

Strategy	p50 (ms)	p95 (ms)	Mean (ms)
No Cache	110.3	113.1	109.5
APC+Retrieval	96.5	103.1	96.1
APC+Optimized	**92.2**	**103.0**	**85.7**

across both model scales and architectures.

Top-k sensitivity. We sweep the number of retrieved documents $k \in \{3, 5, 7, 10\}$ at overlap = 0.6. Ordering improvement is 30.8% at k=5 and grows to 61.2% at k=10 because longer document sequences create more prefix-sharing opportunities. At k=7 the improvement is 7.8%, likely because the overlap-to-k ratio at this point creates fewer long prefix matches. This sweep confirms that larger k (common in production RAG) amplifies the benefit of ordering.

4.8 Geospatial Workload

To test whether ordering benefits persist under a more realistic retrieval pattern, we construct a geospatial workload inspired by Spatial-RAG (Yu et al., 2025) and ITINERA (Tang et al., 2024). We generate 20 "cities" on a geographic grid, each with 10 point-of-interest documents (∼200 tokens). Queries have *spatial locality*: 70% probability of staying near the previous query's city, with documents retrieved from cities within a geographic radius. This creates natural bursty overlap from spatial proximity rather than explicit parameter control.

The geospatial workload has lower average Jaccard similarity (0.17 vs. 0.52 in the synthetic workload) because spatial retrieval produces more diverse document sets. Optimized ordering still reduces TTFT by **4.5%** over retrieval-order APC and **16.4%** over no-cache (Table 10). The smaller gain compared to the synthetic benchmark (4.5% vs. 33%) is expected: with Jaccard = 0.17, consecutive queries share fewer documents, leaving less room for prefix optimization. This result is consistent with the overlap sensitivity analysis (Figure 3), which shows diminishing returns below 0.2 overlap. For spatially concentrated workloads (e.g., a user exploring a single neighborhood), overlap would be higher and ordering gains larger.

4.9 NQ-Open Workload

To evaluate RAGCache++ on a workload derived from real questions, we construct a corpus from the NQ-Open dataset (Yang et al., 2018) using 500 question-answer pairs. We generate topically-clustered Wikipedia-style passages grouped by answer similarity and retrieve top-5 passages per query using TF-IDF similarity. This produces a high-overlap trace (mean Jaccard = 0.88) reflecting concentrated topical retrieval.

Table 11. NQ-Open workload (RTX 4090, 195 queries, mean Jaccard = 0.88).

Strategy	p50 (ms)	p95 (ms)	Mean (ms)	vs. Retr.
No Cache	68.4	70.6	68.2	–
APC+Retrieval	25.7	32.2	27.6	baseline
APC+Optimized	**25.2**	**26.8**	**26.6**	+2.3%

Table 12. Real-corpus evaluation with TF-IDF retrieval (RTX 4090, 200 queries, no controlled overlap).

Strategy	p50 (ms)	p95 (ms)	Mean (ms)	vs. Retr.
No Cache	57.2	67.6	60.5	–
APC+Retrieval	56.2	64.2	54.6	baseline
APC+Optimized	**50.5**	**60.3**	**51.3**	+10.2%

Table 11 shows that at this high-overlap operating point, APC alone already achieves 62% TTFT reduction over no-cache. Ordering adds a modest further 2.3% improvement. This is consistent with the overlap sensitivity analysis (Figure 3): at very high overlap, retrieval order already produces near-maximal prefix alignment. The NQ result confirms that our synthetic overlap sweep generalizes to real-question-derived traces—ordering benefits are concentrated in the moderate-overlap regime.

Mixed-topic workload. To test a workload with heterogeneous, moderate overlap, we generate traces where each query has a 30% probability of switching to a random topic region, producing a mix of within-burst and cross-topic queries (mean Jaccard = 0.31, p50 = 0.43). Optimized ordering reduces p50 TTFT from 94.3 ms to 79.0 ms (**16.3% improvement**), compared with only 4.5% on the geospatial workload (Jaccard = 0.17). This positions the practical benefit: ordering gains scale with the moderate-overlap regime (Jaccard ≈ 0.3–0.5) that characterizes multi-topic RAG deployments where users shift between related subjects.

4.10 Real-Corpus Evaluation

To address the concern that all workloads use synthetic documents, we construct a diverse corpus of 1,000 Wikipedia-style passages spanning 20 topics (machine learning, astronomy, medicine, history, geography, physics, biology, economics, technology, literature, etc.) and use TF-IDF retrieval to create a query trace with *naturally arising* document overlap—no controlled overlap parameter. Table 12 shows that even under very low natural overlap, optimized ordering reduces p50 TTFT by **10.2%** (50.5 ms vs. 56.2 ms). This confirms that the benefit is not an artifact of high controlled overlap: real retrieval patterns produce enough prefix sharing for ordering to help.

Table 13. End-to-end TTFT (ms) including retrieval latency (RTX 4090, 200 queries, 50 output tokens).

T_{ret}	Batch	APC+Retr.	APC+Opt.	Improv.
10 ms	1	69.6	51.8	25.5%
	4	51.8	46.9	9.5%
	8	48.9	45.9	6.3%
30 ms	1	89.7	72.4	19.3%
	4	71.8	67.8	5.6%
	8	68.8	66.8	2.8%
50 ms	1	109.3	92.2	15.7%
	4	91.8	87.4	4.8%
	8	88.7	86.3	2.7%

Table 14. Trie prediction accuracy: correlation between predicted and observed reuse (RTX 4090, 195 queries).

Condition	Pearson r	Spearman ρ	Monotonic?	Zero-pred reuse
Normal (0.90)	0.972	0.925	Yes	3.3%
Pressure (0.78)	0.965	0.925	Yes	2.9%

4.11 End-to-End Pipeline Latency

A practical RAG deployment involves retrieval latency before inference. Table 13 measures end-to-end TTFT (from query arrival through simulated retrieval, prompt construction, and 50-token generation) at three realistic retrieval delays. Even including retrieval overhead, ordering provides **15–26% E2E TTFT reduction** at batch size 1 across all retrieval latencies. Benefits decrease at higher batch sizes (where concurrent prefills amortize the per-request savings) but remain measurable (3–10%).

4.12 Trie Prediction Accuracy

Section 3 notes that the knowledge tree approximates cache residency rather than querying actual vLLM APC state. To validate this approximation, we measure the correlation between the trie's predicted reuse fraction (predicted prefix length / k) and the TTFT-inferred actual reuse fraction ($1 - T_{\text{actual}}/T_{\text{cold}}$). Under normal conditions ($\texttt{gpu_mem} = 0.90$), Pearson $r = 0.972$ and Spearman $\rho = 0.925$; under memory pressure ($\texttt{gpu_mem} = 0.78$), Pearson $r = 0.965$ and Spearman $\rho = 0.925$ (Table 14). The small degradation under pressure confirms that the trie remains an accurate cache proxy even with a smaller KV cache pool.

5 Discussion

Prompt-layer cache scheduling. RAGCache++ introduces *prompt-layer cache scheduling* for APC-based RAG serving, analogous to how disk I/O schedulers reorder requests to exploit spatial locality without modifying the disk firmware. The ordering policy does not modify the cache mechanism itself (APC) but determines which cache lines are populated and in what order, directly controlling the prefill–reuse trade-off. Our profiling confirms this is a genuine systems effect: optimized ordering saves 66% of prefill computation at p50 (Table 6), reduces inference latency by 29% (Table 3), with zero throughput regression and zero memory overhead. The key architectural insight is that a *thin scheduling layer* between retrieval and serving can recover substantial cache locality that the retrieval stage destroys—without requiring engine-level changes. This makes the policy composable with any APC-enabled system and deployable as a drop-in middleware component. We emphasize that the contribution is *cache-aware document ordering as a scheduling abstraction*, not a specific algorithm. Our experiments show that multiple cache-aware heuristics (frequency, PGDSF-proxy, trie-based) all dramatically outperform cache-unaware ordering. The trie variant is preferred for its online adaptivity (Section 4), near-optimal approximation quality (99.7% of oracle), and ability to detect distribution shifts—but the scheduling abstraction itself is the primary insight.

Relation to RAGCache. Our knowledge tree is inspired by RAGCache's (Jin et al., 2025) document-indexed trie structure, but serves a different purpose. RAGCache uses the tree for custom KV cache management within a modified serving engine; we use it solely for ordering decisions, relying on the engine's built-in APC for cache management. Our proxy-baseline experiment (Table 8) shows that both our trie-based ordering and RAGCache-inspired frequency/PGDSF-proxy ordering achieve similar latency reductions (27–30%), confirming that cache-aware ordering—regardless of specific algorithm—is the key mechanism. Our approach is complementary to RAGCache: it achieves comparable scheduling benefits without requiring engine modifications, and could be combined with RAGCache's cache management for further gains.

Relation to CacheBlend. CacheBlend (Yao et al., 2025) addresses a harder problem: reusing KV blocks regardless of position via selective recomputation. Our approach avoids this complexity entirely by converting the problem to pure prefix matching through ordering. The two approaches are complementary: CacheBlend could handle the non-prefix portion of our reordered sequences for further gains.

Limitations.

- **Controlled workloads**: Our main benchmark uses synthetic documents with controlled overlap. The geospatial workload (Section 4.8) provides a more realistic retrieval pattern but still uses synthetic documents; real-corpus validation beyond HotpotQA is needed.
- **Concurrency scope**: Batched evaluation (Table 7) shows ordering benefits at concurrency 1–8, and cache pressure sweeps confirm robustness across memory

levels. However, we do not evaluate under sustained multi-tenant load with realistic arrival distributions (e.g., Poisson arrivals, heterogeneous query patterns), where trie staleness and scheduling interactions may differ.

- **Throughput is decode-dominated**: Ordering improves prefill, not decode. Under workloads with long output sequences (e.g., summarization), the TTFT benefit shrinks as a fraction of total latency.

- **Model family**: We evaluate Qwen2.5 at three scales (0.5B, 1.5B, 7B) and Microsoft Phi-3 (3.8B), observing consistent 18–33% benefits across architectures. Validation on further model families (e.g., Llama, Mistral) remains future work.

Future work. Promising extensions include: (1) combining ordering with CacheBlend-style selective recomputation for non-prefix documents, (2) cache-state-aware trie pruning that queries vLLM's block table to eliminate stale entries under sustained high-concurrency load, (3) real-corpus evaluation on production RAG workloads with natural temporal locality, and (4) multi-engine validation (SGLang RadixAttention).

6 Conclusion

We presented RAGCache++, a prompt-layer scheduling policy that improves KV cache prefix reuse in RAG serving by reordering documents to extend cached prefixes. Under bursty document overlap, RAGCache++ reduces median TTFT by 20–33% over retrieval-order APC across two GPU configurations (statistically significant: 16.8 ± 1.6 ms, 95% CI excludes zero), saving 66% of prefill computation at p50 while adding <0.13% overhead, zero throughput regression, and zero GPU memory cost. End-to-end pipeline measurements confirm that benefits persist when retrieval latency is included (16–26% at $T_{\text{ret}} = 10$–50 ms). The knowledge tree's predicted reuse correlates with observed reuse at $r = 0.97$, validating the approximate cache index design. The greedy ordering achieves 97.5% of oracle performance, benefits persist under concurrent load (7–10% at batch sizes 1–8) and are robust to cache pressure (29–31% across GPU memory levels). On a geospatial workload with naturally lower overlap (Jaccard $= 0.17$), ordering still provides measurable gains (4.5% TTFT reduction). The approach requires no serving-engine modifications ($\sim$150 lines of Python). Our results demonstrate that prefix *depth*—not just hit count—is the key driver of latency reduction in prefix-caching systems.

References

Agarwal, A., Mitra, S., Jain, S., and Gupta, P. Cache-Craft: Managing chunk-caches for efficient retrieval-augmented generation, 2025.

Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., and Wang, H. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2024.

Jin, C., Zhang, Z., Jiang, X., Liu, F., Liu, X., Liu, X., and Jin, X. RAGCache: Efficient knowledge caching for retrieval-augmented generation. *ACM Transactions on Computer Systems*, 44(1):1–27, 2025. doi: 10.1145/3768628.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with PagedAttention, 2023.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., and Kiela, D. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.

Liu, Y., Li, H., Cheng, Y., Ray, S., Huang, Y., Kundu, S., Du, K., Lu, S., and Jiang, J. CacheGen: KV cache compression and streaming for fast large language model serving. *ACM SIGCOMM*, 2024.

Lu, F., Yao, Y., Tong, X., Lin, Y., Zhang, R., Chen, J., and Xie, S. TurboRAG: Accelerating retrieval-augmented generation with precomputed KV caches for chunked text, 2024.

Qwen Team. Qwen2.5 Technical Report. *arXiv preprint arXiv:2412.15115*, 2024.

Tang, Y., Wang, Z., Qu, A., Yan, Y., Wu, Z., Zhuang, D., Kai, J., Hou, K., Guo, X., Zhao, J., Zhao, Z., and Ma, W. ItINERA: Integrating spatial optimization with large language models for open-domain urban itinerary planning. In *EMNLP Industry Track*, 2024.

vLLM Project. Automatic Prefix Caching. https://docs.vllm.ai/en/latest/design/prefix_caching/, 2025. Accessed: 2026-03-31.

Wang, Y., Sheng, Y., Zheng, L., and Stoica, I. ContextPilot: Fast long-context inference via context reuse, 2025.

Yang, Z., Qi, P., Zhang, S., Bengio, Y., Cohen, W. W., Salakhutdinov, R., and Manning, C. D. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018.

Yao, J., Li, H., Liu, Y., Ray, S., Cheng, Y., Zhang, Q., Du, K., Lu, S., and Jiang, J. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys '25)*, Rotterdam, Netherlands, 2025. ACM. doi: 10.1145/3689031.3696098.

Yu, D., Bao, R., Ning, R., Peng, J., Mai, G., and Zhao, L. Spatial-RAG: Spatial retrieval augmented generation for real-world geospatial reasoning questions, 2025.

Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C. H., Cao, S., Kober, C., Shi, C., Zhuang, S., Gonzalez, J. E., Barrett, C., and Stoica, I. SGLang: Efficient execution of structured language model programs, 2024.

A Experimental Configuration Details

Table 15. Hardware and software configuration.

	Config A	**Config B**
GPU	4060 Ti 8 GB	4090 24 GB
Model	Qwen2.5-1.5B	Qwen2.5-7B
vLLM	0.8.5.post1	0.17.1
`max_model_len`	2048	4096
`gpu_mem_util`	0.85	0.90
Corpus	100 docs	500 docs
Queries	100	200
Top-k	5	5
Use	TTFT, overlap	TTFT, systems

B Overlap Sweep Raw Data

Table 16. Raw data for overlap sensitivity experiment (RTX 4060 Ti, Qwen2.5-1.5B-Instruct).

Overlap Frac.	**Jaccard**	**Retrieval p50 (ms)**	**Optimized p50 (ms)**	**Improv.**
0.0	0.386	126.8	111.7	11.9%
0.2	0.386	128.9	114.2	11.5%
0.4	0.417	111.1	77.8	30.0%
0.6	0.526	75.5	60.5	19.9%
0.8	0.661	50.4	50.8	−0.8%

C Systems Benchmark Details

The systems analysis in Section 4.7 (throughput, profiling, pipelining, memory) uses Config B (RTX 4090) with `enforce_eager=True` (CUDA graphs disabled for measurement stability).